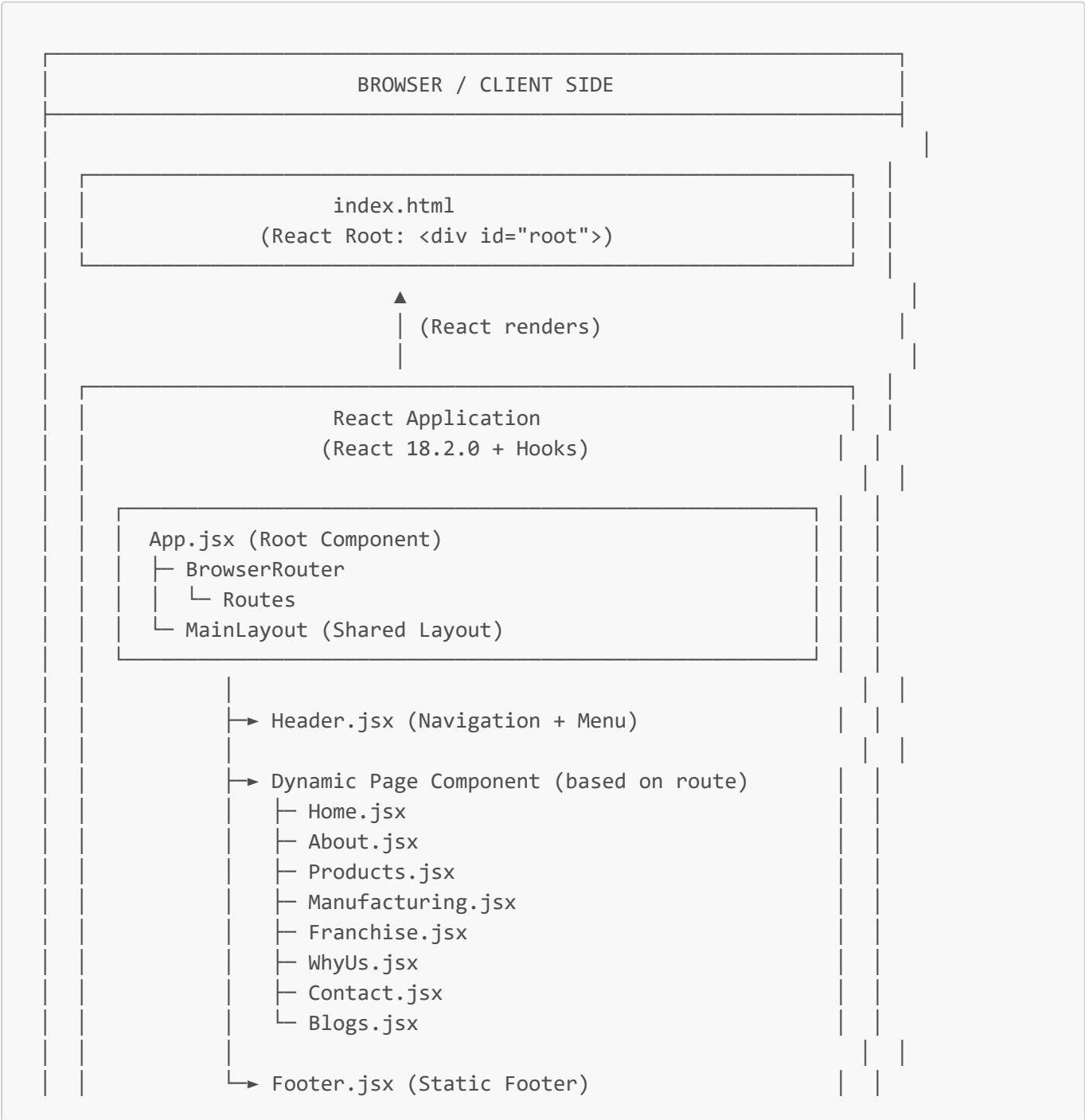


APPLICATION ARCHITECTURE DOCUMENT

1. Executive Summary

Project: Zupharm Laboratories Corporate Website
Architecture Type: Client-Side Rendered (CSR) Single Page Application (SPA)
Architecture Pattern: Component-Based with Functional Hooks
State Management: Local Component State (useState)
Routing: Client-Side Routing (React Router v6)
Deployment Model: Static Site hosted on Netlify CDN

2. High-Level Architecture Diagram





3. Component Hierarchy & Structure

3.1 Application Component Tree

```
App (root with BrowserRouter)
├── Router
│   └── Routes
│       ├── Route path="/" → Home
│       ├── Route path="/about" → About
│       ├── Route path="/products" → Products
│       ├── Route path="/manufacturing" → Manufacturing
│       ├── Route path="/franchise" → Franchise
│       ├── Route path="/why-us" → WhyUs
│       ├── Route path="/contact" → Contact
│       └── Route path="/blogs" → Blogs
└── MainLayout (wrapper for all routes)
    ├── Header
    │   ├── Announcement Bar (Layer 1)
    │   └── Brand Strip (Layer 2)
    │       ├── Logo
    │       └── Navigation Pill
```

```
| |   └─ NAV_ITEMS map (About, Products, Franchise, etc.)
| |   └─ Mobile Hamburger Menu
| |
| └─ main (children - dynamic page content)
|   └─ [Current Page Component]
|
| └─ Footer
|   └─ Brand Section
|   └─ Navigation Links
|   └─ Products Links
|   └─ Knowledge Centre
|   └─ Contact Info
|   └─ Copyright & Legal
```

3.2 Page-Specific Component Breakdown

Home.jsx (Landing Page)

```
Home
└─ useCounter() [Custom Hook]
  └─ Animated counter for stats
└─ StatsBar (component)
  └─ useCounter x3 (products, states, partners)
  └─ useIntersectionObserver [for scroll trigger]
    └─ stat-cell x4 [rendered inline]
└─ Hero Section
  └─ Hero text (tagline, CTA buttons)
  └─ Hero visual (image, float stat)
    └─ StatsBar
└─ Bento Capabilities Section
  └─ 4 Bento cards (WHO-GMP, Pan-India, R&D, AI)
    └─ Icons and descriptions
└─ Products Teaser Section
  └─ FEATURED_PRODUCTS.map() → 3 product cards
└─ Certification Strip
  └─ Logos and certifications
```

About.jsx (Company Information)

```
About
└─ useState → selectedCert (certification modal state)
└─ Certification Modal
  └─ onClick handlers for cert details
└─ Page Hero Section
└─ Vision & Mission Section
└─ Core Values Grid
  └─ VALUES.map() → value-card-zu x6
```

- └─ Certifications Grid
 - └─ CERTS.map() → certification cards with modal trigger

Products.jsx (Product Portfolio)

```
Products
├─ Page Hero
├─ Products Section
│   └─ PRODUCTS.map() → prod-card x8
│       ├── prod-img
│       ├── prod-count
│       ├── prod-name
│       ├── prod-desc
│       └─ prod-link
└─ Footer call-to-action
```

Manufacturing.jsx (Facilities & Capabilities)

```
Manufacturing
├─ useState → activeTab, openFaq, openResearch
├─ Page Hero
├─ Tabs Component
│   ├── activeTab state management
│   └─ TABS.map() → tab content (Sterile, Solids, R&D)
├─ FAQ Accordion Section
│   ├── openFaq state
│   └─ FAQ.map() → accordion items
└─ Research Capabilities
    ├── openResearch state
    └─ R&D capabilities display
```

Franchise.jsx (PCD Franchise Page)

```
Franchise
├─ useState → form (multi-field form state)
├─ handleChange() [form input handler]
├─ handleSubmit() [form submission]
├─ Page Hero
├─ Franchise Left Column
│   ├── Text content
│   └─ USPS.map() → usp-row x3
│       ├── usp-num
│       ├── usp-content
│       └─ Title + description
├─ Franchise Form Right Column
│   └─ 7 form fields
```

- | | Select dropdown (STATES)
- | | Submit button
- | Call-to-action

Contact.jsx (Contact & Inquiry)

- Contact
 - | useState → form, submitted
 - | INQUIRY_TYPES (6 types)
 - | WhatsAppHoverButton() [floating button component]
 - | Page Hero
 - | Contact Form Card
 - | Conditional render (submitted or form)
 - | form-row x3
 - | Form fields (name, phone, email, inquiry type, message)
 - | Submit button
 - | Contact Info Cards (Address, Phone, Email)
 - | Submission Success Message

Blogs.jsx (Knowledge Centre)

- Blogs
 - | useState → selectedBlog, isMobile
 - | useEffect → window resize listener
 - | BLOG_DATA (6 blog posts)
 - | Page Hero (dynamic - changes based on selectedBlog)
 - | Conditional Render:
 - | View A: Blog List (3-column grid)
 - | | BLOG_DATA.map() → blogCustomCard x6
 - | | | blog image
 - | | | blog title
 - | | | blog excerpt
 - | | | onClick → setSelectedBlog()
 - | View B: Expanded Blog View
 - | | Back button
 - | | Blog image
 - | | Meta info (author, date)
 - | | contentParagraphs.map() → paragraphs

WhyUs.jsx (Competitive Advantages)

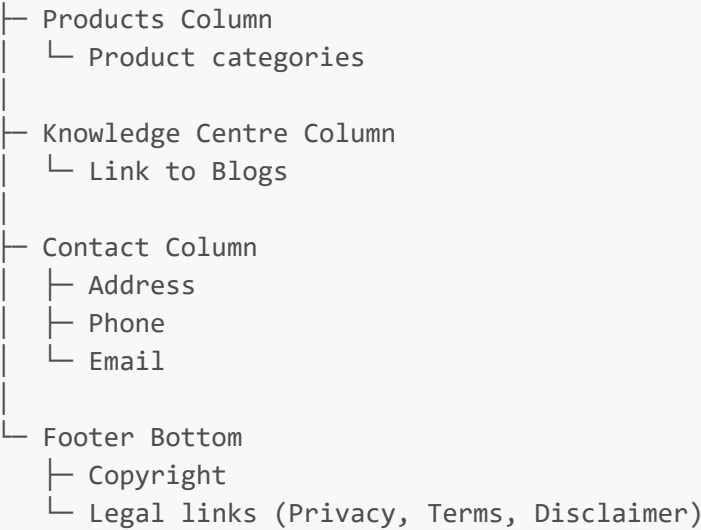
- WhyUs
 - | Why Grid Section
 - | | REASONS.map() → why-item x6
 - | | | why-item-num

- ├─ Title
- ├─ Description
- └─ AI Tools Section
 - ├─ AI_TOOLS.map() → ai-tool x4 (ZuAI, ZuDoc, ZuSupply, ZuAnalytics)
 - ├─ ai-tool-img
 - ├─ ai-badge
 - ├─ Title
 - └─ Description

3.3 Layout Component Hierarchy

MainLayout (Functional Component)

- ├─ Header
 - ├─ Layer 1: Announcement Bar
 - ├─ Announcement dot
 - ├─ Text content
 - └─ Close button (toggles visibility)
 - ├─ Layer 2: Brand Strip
 - ├─ Logo (Link to home)
 - ├─ Navigation Pill
 - ├─ NAV_ITEMS
 - ├─ About
 - ├─ Products
 - ├─ Franchise
 - ├─ Manufacturing
 - └─ Contact
 - ├─ Apply CTA Button
 - └─ Hamburger (Mobile Menu Toggle)
 - └─ Mobile Drawer (conditional render)
 - ├─ Backdrop
 - ├─ Nav links (copy of desktop nav)
 - └─ Close button
 - └─ Layer 3: Header Shadow (on scroll)
- ├─ main element
 - └─ {children} - Current Page Component
- └─ Footer
 - ├─ Brand Column
 - ├─ Logo
 - ├─ Company description
 - └─ Social icons (Facebook, LinkedIn, Instagram, YouTube)
 - ├─ Navigation Column
 - └─ Links to all pages



4. Routing Structure (React Router v6)

4.1 Route Configuration

```
// src/App.jsx
<Router>
  <MainLayout>
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/about" element={<About />} />
      <Route path="/products" element={<Products />} />
      <Route path="/manufacturing" element={<Manufacturing />} />
      <Route path="/franchise" element={<Franchise />} />
      <Route path="/why-us" element={<WhyUs />} />
      <Route path="/contact" element={<Contact />} />
      <Route path="/blogs" element={<Blogs />} />
    </Routes>
  </MainLayout>
</Router>
```

4.2 Route Mapping

Route	Component	Purpose	Nested Routes
/	Home	Landing page with hero, stats, products	None
/about	About	Company vision, mission, values, certifications	None
/products	Products	12 therapeutic product categories	None
/manufacturing	Manufacturing	Facilities, tabs, FAQs, certifications	None
/franchise	Franchise	PCD franchise form, USPs, benefits	None

Route	Component	Purpose	Nested Routes
/why-us	WhyUs	6 reasons + AI tools showcase	None
/contact	Contact	Contact form, WhatsApp, inquiry types	None
/blogs	Blogs	Knowledge centre, 6 blog posts	None (dynamic blog view)

4.3 Client-Side Navigation

Using React Router Links:

```
// Navigation
<NavLink to="/" activeClassName="active">Home</NavLink>
<Link to="/about">About Us</Link>
<Link to="/products" className="btn-main">Explore Products</Link>
<Link to="/franchise" className="btn-ghost">Apply for Franchise</Link>
```

Navigation Triggers:

- Header navigation links
- Footer navigation links
- Inline CTAs on pages
- Back buttons in modal/expanded views (Blogs.jsx)

5. State Management Approach

5.1 Current State Management Strategy

Model: Local Component State (Functional Hooks)
Complexity: No global state management (No Redux, Context API, or Zustand)
Scaling: Works well for MVP; consider Context API or Redux for Phase 2+

5.2 State Management by Component

Component	State Variables	Purpose	Scope
Header	showAnnouncement, menuOpen	Announcement visibility, mobile menu	Local
About	selectedCert	Certification modal selection	Local
Manufacturing	activeTab, openFaq, openResearch	Tab selection, FAQ/Research accordion state	Local
Franchise	form (object)	Form field values	Local
Contact	form (object), submitted	Form data, submission status	Local

Component	State Variables	Purpose	Scope
Blogs	selectedBlog, isMobile	Blog selection, responsive state	Local
Home	visible (in StatsBar)	Scroll-triggered animation	Local
CertificationCarousel	currentIndex, visibleCount	Carousel position, visible items	Local
Accordion	openItemId	Open accordion item	Local
Tabs	activeTab	Active tab selection	Local

5.3 Custom Hooks

useCounter (Home.jsx)

```
function useCounter(target, duration = 1800, start = false) {
  const [count, setCount] = useState(0);
  useEffect(() => {
    // Animation logic with requestAnimationFrame
    // Eases from 0 to target over duration ms
  }, [target, duration, start]);
  return count;
}
```

Purpose: Animate stat counters on scroll trigger

5.4 State Flow Example: Contact Form



6. External Libraries & Their Purposes

6.1 Core Libraries

Library	Version	Purpose	Usage
React	18.2.0	UI library, component rendering	Entire application
ReactDOM	18.2.0	DOM rendering for React	Application root mount
React Router DOM	6.22.0	Client-side routing, navigation	App.jsx, all pages, Header, Footer

6.2 Styling & UI

Library	Version	Purpose	Usage
Tailwind CSS	3.4.0	Utility-first CSS framework	All styling (classes in JSX)
PostCSS	8.4.0	CSS transformation pipeline	Processes Tailwind directives
Autoprefixer	10.4.0	Vendor prefix auto-injection	Browser compatibility

6.3 Animations & Motion

Library	Version	Purpose	Usage
AOS	3.0.0-beta.6	Animate On Scroll library	Scroll-triggered animations (initialized in main.jsx)
Framer Motion	10.12.16	Advanced motion library	Complex animations, transitions (if used)

6.4 SEO & Metadata

Library	Version	Purpose	Usage
React Helmet Async	1.3.0	Manage document head (SEO)	Page title, meta tags, Open Graph

6.5 Build & Development Tools

Tool	Version	Purpose	Usage
Vite	5.0.0	Next-gen build tool	Development server, production build
@vitejs/plugin-react	4.0.0	React support for Vite	Fast Refresh, JSX transformation

6.6 Testing & Quality

Tool	Version	Purpose	Usage
Jest	29.0.0	Unit testing framework	Component tests (not yet implemented)
@testing-library/react	14.0.0	React component testing	User interaction testing
Cypress	13.0.0	E2E testing framework	Full workflow testing (not yet implemented)

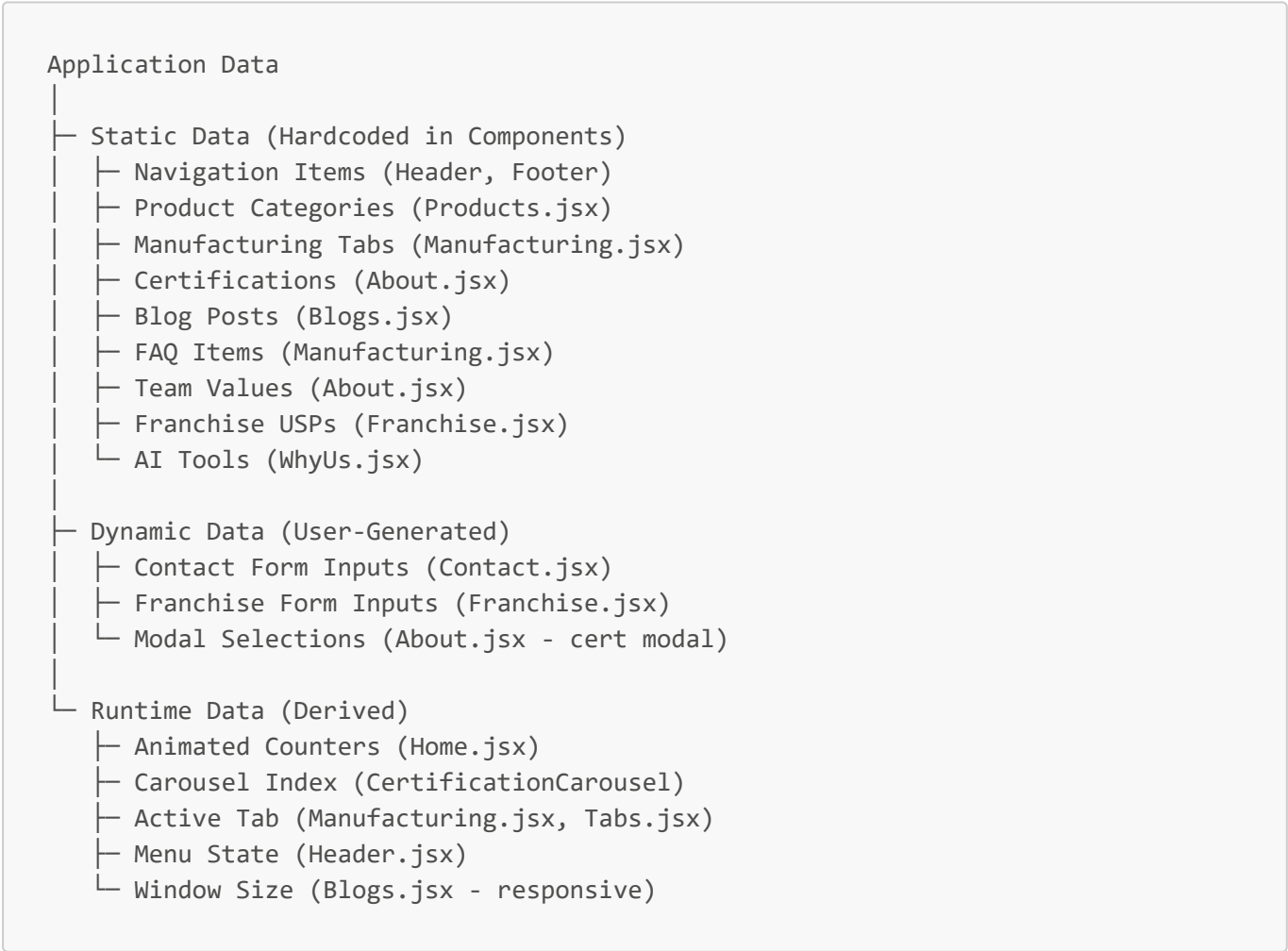
Tool	Version	Purpose	Usage
ESLint	8.0.0	Code quality linter	Code style enforcement
eslint-plugin-react	7.0.0	React-specific linting	React best practices
axe-core	4.7.0	Accessibility testing	WCAG compliance checking

6.7 External Assets & CDNs

Resource	Purpose	Source
Google Fonts	Typography (DM Serif Display, DM Sans)	fonts.googleapis.com
Font Awesome Icons	Icon library (v6.5.0)	cdnjs.cloudflare.com
Unsplash Images	Product & feature images	images.unsplash.com
Cloudflare CDN	Global content delivery	Netlify integration

7. Data Flow Architecture

7.1 Information Architecture (IA)



7.2 Form Data Flow Example

Franchise Form Data Structure:

```
{
  name: string,
  mobile: string,
  email: string,
  city: string,
  product: string,
  businessType: string,
  message: string,
  state: enum (STATES array)
}
```

Flow:

User fills form → handleChange updates form state

↓

(Form re-renders with new values)

↓

User submits

↓

handleSubmit triggered

↓

Form data ready for API (future)

↓

Currently: Reset form (future enhancement)

8. API Integration Points (Future)

Current Status: No backend API integration yet

Planned Integration Points:

Contact Form (Contact.jsx)

- POST /api/inquiries
- Save to database

Franchise Form (Franchise.jsx)

- POST /api/franchise-applications
- Save to CRM

Blog Data (Blogs.jsx)

- GET /api/blogs
- Replace hardcoded BLOG_DATA

Blog Comments (Future)

- POST /api/blogs/:id/comments
- User interaction

Lead Tracking

- POST /api/analytics
- Track form submissions, page views

9. Performance Considerations

9.1 Optimization Strategies Implemented

Strategy	Implementation	Benefit
Code Splitting	React Router lazy loading (future)	Reduce initial bundle
Image Optimization	Unsplash URLs with <code>w=</code> parameter	Responsive image sizes
Memoization	useCallback/useMemo (future)	Prevent unnecessary re-renders
CSS in JS	Tailwind CSS (utility classes)	No unused CSS in bundle
CDN Caching	Netlify Edge + Cloudflare	Fast global delivery

9.2 Lighthouse Metrics Target

- **Performance:** > 90
- **Accessibility:** > 95
- **Best Practices:** > 90
- **SEO:** > 95

10. Security Considerations

10.1 Current Security Measures

Aspect	Implementation	Status
XSS Protection	React auto-escapes JSX	✅ Implemented
HTTPS	Netlify SSL/TLS	✅ Implemented
Environment Variables	.env files (future)	⌚ To implement
Input Validation	Client-side (future backend)	⌚ To implement
CSRF Protection	Needed for forms	⌚ To implement
API Authentication	JWT (future)	⌚ To implement

11. Browser Compatibility

Supported Browsers:

- Chrome/Chromium v90+
- Firefox v88+
- Safari v14+

- Edge v90+
- Mobile browsers (iOS Safari, Chrome Android)

Responsive Breakpoints:

- Mobile: 320px - 767px
- Tablet: 768px - 1023px
- Desktop: 1024px - 1919px
- Large Desktop: 1920px+

12. Deployment Architecture

```
GitHub Repository
  ↓
Netlify (Auto Deploy)
  ↓
├ Build: npm run build → dist/
├ Optimization: Minify, compress
├ Cache Headers: Set via netlify/_headers
├ CDN: Cloudflare global edge network
  ↓
Static Assets Cached at Edge
  ↓
User Browser (Global)
```

13. Scalability & Future Enhancements

Phase 2 Recommendations

1. State Management:

- Implement Context API for global settings (theme, language)
- Consider Redux Toolkit for complex state (future API integration)

2. API Integration:

- Backend Express/Node.js server
- PostgreSQL/MongoDB database
- RESTful API endpoints for forms, blog, products

3. Performance:

- Lazy load route components
- Implement React.memo for expensive components
- Add error boundary components

4. Testing:

- Set up Jest unit tests
- Add Cypress E2E tests
- Implement CI/CD with GitHub Actions

5. **Features:**

- User authentication
- Product filtering & search
- Admin dashboard
- Analytics tracking
- Email notifications

14. Architecture Decision Log

Decision	Reason	Impact
No global state mgmt	MVP simplicity	Works now; needs refactor if complex
Client-side routing	SPA performance	No page refreshes; better UX
Tailwind CSS	Rapid development	Small CSS footprint, high customization
Static data in components	Quick MVP development	Easy to migrate to API later
React Hooks over classes	Modern best practices	Simpler code, better maintainability

Document Version: 1.0
Last Updated: June 10, 2026
Maintained By: Farhat Afreen & Development Team